

Assignment 2

Required Tools: GCC (GNU Compiler Collection): Required to compile C programs. Install with:
sudo apt install gcc

```
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform: ~/Desktop
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~/Desktop$ sudo apt install gcc
[sudo] password for ahmedelhagey:
Sorry, try again.
[sudo] password for ahmedelhagey:
Reading package lists... Done
Building dependency tree... Done
```

Task 1

Exercise 1: Using fork() in C Write a C program that demonstrates process creation using the fork() system call.

```
GNU nano 7.2 p.c *
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform: ~
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("This is the child process. PID: %d\n", getpid());
    } else if (pid > 0) {
        printf("This is the parent process. PID: %d\n", getpid());
    } else {
        printf("Forkfailed!\n");
    }
    return 0;
}
```

```

ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ nano p.c
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ gcc p.c -o out
p.c:2:19: error: missing terminating > character
  2 | #include <unistd.h
    |           ^
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ nano p.c
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ gcc p.c -o out
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ ./out
This is the parent process. PID: 6618
This is the child process. PID: 6619

```

Start and Stop a Process

Exercise 2: Starting Processes in the Background

Exercise 3: Stopping Processes

Exercise 4: Pausing and Resuming a Process

```

ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ sleep 100 &
[1] 6625
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ sleep 300 &
[2] 6629
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ jobs
[1]-  Running                  sleep 100 &
[2]+  Running                  sleep 300 &
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ kill -STOP 6629
[1]   Done                    sleep 100

[2]+  Stopped                  sleep 300
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ jobs
[2]-  Stopped                  sleep 300
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ kill -CONT 6629
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ jobs
[2]+  Running                  sleep 300 &
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ kill 6629
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ jobs
[2]+  Terminated              sleep 300
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$

```

```

ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ gcc p.c -o out
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ ./out
This is the parent process. PID: 6755
This is the child process. PID: 6756
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ sleep 300 &
[1] 6757
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ jobs
[1]+  Running                  sleep 300 &
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ ps aux | grep sleep
ahmedel+   6757  0.0  0.0  8288  1940 pts/0    S   22:02   0:00  sleep 300
ahmedel+   6760  0.0  0.0  9144  2248 pts/0    S+  22:03   0:00  grep --color=
auto sleep
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ kill 6757
[1]+  Terminated              sleep 300
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ ps aux | grep sleep
ahmedel+   6762  0.0  0.0  9144  2248 pts/0    S+  22:03   0:00  grep --color=
auto sleep
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ █

```

Task 2

Exercise 5: Role of the Linker

File 1.c

```

GNU nano 7.2 file1.c
#include <stdio.h>
int main() {
printf("This is a simple program.\n");
return 0;
}

```

File2.c

```
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform: ~$ nano file1.c
void hello();
int main() {
hello();
return 0;
}

ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ nano file2.c
14 GB Volume
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ gcc file1.c file2.c -o out
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ ./out
Hello from file1!
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$
```

Task 3

Exercise 6: Role of the Loader Write a simple program and inspect the libraries it uses with ldd Simple Program:

```
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform: ~$ nano simple.c
GNU nano 7.2
#include <stdio.h>
int main() {
printf("This is a simple program.\n");
return 0;
}
```

```

ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ nano simple.c
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ gcc simple.c -o out
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ ./out
This is a simple program.
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ ldd simple.c
        not a dynamic executable
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$ ldd out
        linux-vdso.so.1 (0x0000762fdf2a6000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000762fdf000000)
        /lib64/ld-linux-x86-64.so.2 (0x0000762fdf2a8000)
ahmedelhagey@ahmedelhagey-VMware-Virtual-Platform:~$

```

Make file

CC = gcc

CFLAGS = -Wall -Wextra -g

all: task1 task2 task3

task1: p.c

\$(CC) \$(CFLAGS) p.c -o task1

Task2: file 1.c file2.c

\$(CC) \$(CFLAGS) file 1.c file2.c -o task2

Task3: simple.c

\$(CC) \$(CFLAGS) simple.c -o task3

clean:

rm -rf task1 task2 task3